

FE TechSpec

문서 메타

항목	내용
문서	FE TechSpec — gohae v1.0
상위 문서	기획문서
버전	v1.2 (리뷰 라운드 2 반영)
최종 업데이트	@2026년 5월 16일
작성자	오준경 (jjuney)
대상 독자	본인 (1인 개발) — 12시간 스프린트 중 구현 가이드



스코프 — gohae v1.0의 FE(Next.js 풀스택의 클라이언트·서버 컴포넌트 + Server Action 레이어 + UI) 구현 명세. 데이터 모델·API 본체·외부 통합은 [BE TechSpec](#) 참조. **Drizzle 호출은 RSC·Server Action에서도 일어남** (Next.js 풀스택 — FE/BE 경계는 "관심사"로 갈림).



v1.1 → v1.2 변경 요지 — (1) `(app)/layout.tsx` 에서 persona 조회 제거(결정 로 그와 일치, cache 우회 회피). (2) 위기 신호 영속화: `useChat.data` 의존 제거 → `crisisFlaggedIds: Set<string>` 별도 state. (3) `endConversationAction` 의 fetch 함정 코드 삭제 → `extractAndStoreMemories` 직접 호출. (4) 새 대화 `conversationId` 갱신: 응답 헤더 `x-conversation-id` + `useState` 동기화. (5) `getCurrentPersona` 의 dynamic import → 정적 import. (6) `crisisActive` reset 시점 명세(새 user 메시지 submit). (7) `shadcn alert` 설치 목록에서 제거. (8) Sentry exception `value` scrubbing + "에러 메시지에 사용자 입력 인 터플레이트 금지" 규칙. (9) Drizzle 쓰는 모든 route handler `runtime = 'nodejs'` cross-link.

1. 기술 스택

영역	선택	버전·비고
Framework	Next.js 15 (App Router)	RSC 우선, Server Actions

영역	선택	버전·비고
Language	TypeScript	strict 모드
Styling	Tailwind CSS v4	shadcn/ui와 호환
UI Kit	shadcn/ui (new-york)	복붙 컴포넌트, 직접 소유
채팅 UI	Vercel AI SDK (<code>ai</code> , <code>@ai-sdk/react</code>)	<code>useChat</code> 혹
Form/Validation	react-hook-form + zod	온보딩 폼
Auth Client	@supabase/ssr	SSR 친화
ORM (FE/BE 공유)	Drizzle ORM	RSC·Server Action에서도 직접 호출
Icons	lucide-react	shadcn 기본
Toast/Notification	sonner	shadcn 권장
Routing helper	next/navigation	<code>useRouter</code> , <code>redirect</code>
Font	Noto Sans KR (next/font/google)	한국어 가독성
에러 모니터링	@sentry/nextjs	FE/BE 통합, exception scrubbing 포함



Runtime 규칙 — Drizzle (postgres-js)을 사용하는 모든 Route Handler·Server Action 파일은 `export const runtime = 'nodejs'` 명시. Edge runtime에서 postgres-js 미동작. 대상: `/api/chat` , `/api/conversation/end` , `/auth/callback` , 모든 Server Action 파일. BE TechSpec §1·§6 동일 규칙.

2. 폴더 구조

```

app/
├── (marketing)/
│   ├── page.tsx           # 랜딩 /
│   ├── login/page.tsx    # /login
│   ├── terms/page.tsx    # /terms
│   └── privacy/page.tsx   # /privacy
├── (app)/
│   └── layout.tsx         # 로그인 필요 영역
└──                         # ★ 인증만 – persona 분기는 pa
ge에서

```

```

|   |─ onboarding/
|   |   |─ page.tsx           # /onboarding (persona 있으면
/chat redirect)
|   |   └─ actions.ts        # Server Action: persona 생성
|   |       └─ chat/
|   |           |─ page.tsx   # /chat (RSC: 데이터 fetch만)
|   |           └─ chat-client.tsx # 'use client' – useChat + st
ate
|   |               └─ actions.ts # 대화 종료 (메모리 추출 직접 호
출)
|   └─ api/
|       |─ chat/route.ts      # POST – Claude 스트리밍
|       └─ health/route.ts   # GET
|   └─ auth/callback/route.ts # OAuth callback (persona 분
기)
|   └─ layout.tsx            # root layout (fonts, Sentry,
sonner)
|   └─ global-error.tsx
|   └─ error.tsx
|   └─ not-found.tsx

components/
|   └─ ui/                   # shadcn (Alert 제외)
|   └─ chat/
|       |─ message-list.tsx
|       |─ message-bubble.tsx
|       |─ crisis-card.tsx   # custom div (shadcn Alert 미
사용)
|       |─ chat-input.tsx
|       |─ empty-state.tsx
|       └─ end-conversation-button.tsx
|   └─ onboarding/persona-form.tsx
|   └─ marketing/{hero, footer}.tsx

lib/
|   └─ supabase/{client, server, middleware}.ts
|   └─ db/                   # Drizzle (BE TechSpec와 공유)
|   └─ ai/                   # BE TechSpec

```

```

├─ crisis/keywords.ts      # FE/BE 공유
├─ auth.ts                # getCurrentUser, getCurrentP
ersona (cache)
└─ utils.ts

middleware.ts              # Edge: 세션 refresh + 비로그인
차단만

instrumentation.ts        # Sentry init
sentry.{client,server,edge}.config.ts

```

3. 라우팅 & 접근 제어



책임 분기 — `middleware.ts` 는 Edge runtime이라 **DB 조회 금지**.
`(app)/layout.tsx` 는 **인증만** (persona 미조회). persona 유무 분기는 각 page
RSC에서 `getCurrentPersona()` 호출.

3.1 가드 매트릭스

경로	인증	persona 분기	가드 위치
<code>/</code> (랜딩)	✗	—	—
<code>/login</code>	✗	—	—
<code>/terms</code> , <code>/privacy</code>	✗	—	—
<code>/auth/callback</code>	✗	persona 조회 후 <code>/chat</code> or <code>/onboarding</code>	route handler
<code>/onboarding</code>	✓	persona 있으면 <code>/chat</code> redirect	<code>onboarding/page</code> RSC
<code>/chat</code>	✓	persona 없으면 <code>/onboarding</code> redirect	<code>chat/page</code> RSC
<code>/api/health</code>	✗	—	—
<code>/api/chat</code>	✓	—	route handler 내부 <code>getCurrentUser()</code>

3.2 `middleware.ts`

```
// middleware.ts
import { NextResponse, type NextRequest } from 'next/serve
r'
import { updateSession } from '@lib/supabase/middleware'

export async function middleware(req: NextRequest) {
  const { response, user } = await updateSession(req)
  const path = req.nextUrl.pathname

  const requiresAuth = path.startsWith('/chat') || path.sta
rtsWith('/onboarding')
  if (requiresAuth && !user) {
    return NextResponse.redirect(new URL('/login', req.ur
l))
  }
  return response
}

export const config = {
  matcher: ['/(?!_next/static|_next/image|favicon.ico|.*
\\.?(?:png|svg|ico)$).*)'],
}
```

3.3 (app)/layout.tsx — 인증만

```
// app/(app)/layout.tsx
import { redirect } from 'next/navigation'
import { getCurrentUser } from '@lib/auth'

export default async function AppLayout({ children }: { chi
ldren: React.ReactNode }) {
  const user = await getCurrentUser()
  if (!user) redirect('/login')
  return <>{children}</>
}
```

persona 조회 없음. layout이 persona를 조회해서 사용하지 않으면 cache 우회·의미 없는 DB 호출. persona 분기는 각 page에서 `getCurrentPersona()` 호출 (React `cache` 로 같은 request 내 중복 호출 방지).

4. 디자인 시스템 — Tailwind 토큰

4.1 `app/globals.css`

```
@import "tailwindcss";

@theme {
  --color-bg-canvas: #FAFAF7;
  --color-bg-surface: #FFFFFF;
  --color-bg-subtle: #F4F2EE;
  --color-text-primary: #1A1A1A;
  --color-text-secondary: #5C5C5C;
  --color-text-muted: #9A9A9A;
  --color-border: #E5E2DC;
  --color-accent: #2D4A4A;
  --color-accent-hover: #1F3535;
  --color-accent-fg: #FFFFFF;
  --color-danger: #BF3D3D;
  --color-danger-bg: #FCEEEE;
  --radius-sm: 4px;
  --radius-md: 8px;
  --radius-lg: 12px;
  --radius-xl: 16px;
  --font-sans: var(--font-noto-sans-kr), system-ui, sans-serif;
}

body { background: var(--color-bg-canvas); color: var(--color-text-primary); font-family: var(--font-sans); }
html, body { height: 100%; }
```

4.2 shadcn `components.json`

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "new-york",
  "rsc": true,
  "tsx": true,
  "tailwind": { "css": "app/globals.css", "baseColor": "neutral", "cssVariables": true },
  "aliases": {
    "components": "@components", "utils": "@lib/utils",
    "ui": "@components/ui"
  },
  "iconLibrary": "lucide"
}
```

4.3 v1.0에 필요한 shadcn 컴포넌트

`npx shadcn@latest add button input textarea card form label select avatar separator sonner checkbox`

alert 제거 — `<CrisisCard />` 는 custom div(\$9.2)로 작성. shadcn Alert 미사용. 단 순성 원칙.

5. 상태 관리 & 데이터 액세스



원칙 — RSC + Server Actions 우선. 클라이언트 상태는 채팅 UI(`useChat`)와 폼(`react-hook-form`)에만. 글로벌 상태 라이브러리 금지. **DB 호출은 RSC와 Server Action에서 Drizzle로 직접** (스트리밍 필요한 `/api/chat` 만 예외).

상태	위치	도구
인증 세션	Server (cookies, RSC)	Supabase <code>getUser()</code> (React <code>cache</code>)
페르소나	Server (DB 직접)	RSC + Drizzle (<code>getCurrentPersona</code>)
대화 메시지	Client (useChat) + Server (영속화)	Vercel AI SDK

상태	위치	도구
활성 conversationId	Client <code>useState</code> (헤더로 동기화)	<code>x-conversation-id</code> 응답 헤더
위기 카드 표시 (영속 + 활성)	Client <code>crisisFlaggedIds: Set<string></code>	history → 초기화, stream 도착 시 추가
위기 활성 신호 (a11y alert용)	Client <code>crisisActive: boolean</code>	<code>x-crisis</code> 헤더
입력 폼	Client	react-hook-form + zod
토스트	Client	sonner

5.1 인증/페르소나 헬퍼 — 정적 import

```
// lib/auth.ts
import { cache } from 'react'
import { eq } from 'drizzle-orm'
import { createClient } from './supabase/server'
import { db } from './db'
import { personas } from './db/schema'

export const getCurrentUser = cache(async () => {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  return user
})

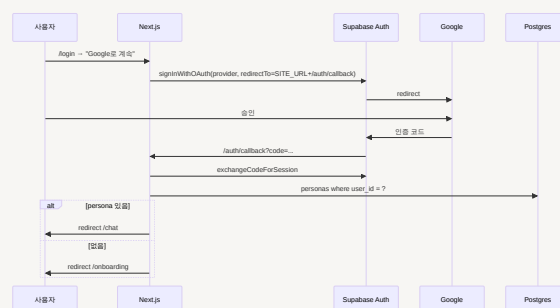
export const getCurrentPersona = cache(async () => {
  const user = await getCurrentUser()
  if (!user) return null
  return (await db.query.personas.findFirst({
    where: eq(personas.userId, user.id)
  })) ?? null
})
```

dynamic import 제거. tree-shaking은 정적 import에서 동일하게 동작. 가독성·유지 보수 우선.

6. 인증 흐름 (Supabase Google OAuth)

6.1 흐름도

```
sequenceDiagram
    participant U as 사용자
    participant FE as Next.js
    participant SB as Supabase Auth
    participant G as Google
    participant DB as Postgres
    U->>FE: /login → "Google로 계속"
    FE->>SB: signInWithOAuth(provider, redirectTo=SITE_URL
    +/auth/callback)
    SB->>G: redirect
    U->>G: 승인
    G->>SB: 인증 코드
    SB->>FE: /auth/callback?code=...
    FE->>SB: exchangeCodeForSession
    FE->>DB: personas where user_id = ?
    alt persona 있음
        FE->>U: redirect /chat
    else 없음
        FE->>U: redirect /onboarding
    end
end
```



6.2 코드

login 페이지

```
// app/(marketing)/login/page.tsx
'use client'
import { createClient } from '@lib/supabase/client'
import { Button } from '@components/ui/button'

export default function LoginPage() {
  const supabase = createClient()
  const handleGoogle = async () => {
    const siteUrl = process.env.NEXT_PUBLIC_SITE_URL ?? window.location.origin
    await supabase.auth.signInWithOAuth({
      provider: 'google',
      options: { redirectTo: `${siteUrl}/auth/callback` }
    })
  }
  return (
    <div className="flex h-dvh items-center justify-center">
      <Button onClick={handleGoogle}>Google로 계속</Button>
    </div>
  )
}
```

OAuth callback — persona 분기

```
// app/auth/callback/route.ts
import { NextResponse } from 'next/server'
import { eq } from 'drizzle-orm'
import { createClient } from '@lib/supabase/server'
import { db } from '@lib/db'
import { personas } from '@lib/db/schema'

export const runtime = 'nodejs' // Drizzle (postgres-js) 사용

export async function GET(req: Request) {
  const url = new URL(req.url)
```

```

const code = url.searchParams.get('code')
const origin = process.env.NEXT_PUBLIC_SITE_URL ?? url.origin

if (!code) return NextResponse.redirect(new URL('/login?error=missing_code', origin))

const supabase = await createClient()
const { error } = await supabase.auth.exchangeCodeForSession(code)
if (error) return NextResponse.redirect(new URL('/login?error=oauth', origin))

const { data: { user } } = await supabase.auth.getUser()
if (!user) return NextResponse.redirect(new URL('/login?error=no_user', origin))

const persona = await db.query.personas.findFirst({
  where: eq(personas.userId, user.id)
})

// depth-in-defense: callback + onboarding/page 둘 다 가드
return NextResponse.redirect(new URL(persona ? '/chat' : '/onboarding', origin))
}

```

7. 화면별 상세 명세

7.1 랜딩 /

비로그인 진입점. 헤더(로고 + 로그인 링크) → Hero("나를 기억하는 대화" + CTA → `/login`) → 차별점 2카드 → 동작 3step → Footer(약관/개인정보/1393). 완전 RSC.

7.2 로그인 `/login`

단일 "Google로 계속" 버튼. `?error=` query 있으면 sonner toast.

7.3 온보딩 /onboarding

가드 (재방문 보호)

```
// app/(app)/onboarding/page.tsx
import { redirect } from 'next/navigation'
import { getCurrentPersona } from '@lib/auth'
import { PersonaForm } from '@components/onboarding/persona-form'

export default async function OnboardingPage() {
  const persona = await getCurrentPersona()
  if (persona) redirect('/chat')

  return (
    <div className="mx-auto max-w-md py-12">
      <h1 className="text-2xl font-bold mb-6">잠깐만요, 알려주세요</h1>
      <PersonaForm />
    </div>
  )
}
```

폼 schema

```
// app/(app)/onboarding/schema.ts
import { z } from 'zod'

export const personaSchema = z.object({
  nickname: z.string().min(1, '닉네임을 입력해주세요').max(20),
  ageRange: z.enum(['10s', '20s', '30s', '40s+']),
  tone: z.enum(['formal', 'casual']).default('formal'),
  selfIntro: z.string().max(500).optional(),
  agreeTerms: z.literal(true, {
    errorMap: () => ({ message: '약관·개인정보 동의가 필요합니다' })
  })
})
```

```

}))
export type PersonaInput = z.infer<typeof personaSchema>

```

폼 컴포넌트

```

// components/onboarding/persona-form.tsx
'use client'
import { useRouter } from 'next/navigation'
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { toast } from 'sonner'
import { personaSchema, type PersonaInput } from '@app/(app)/onboarding/schema'
import { createPersonaAction } from '@app/(app)/onboarding/actions'

export function PersonaForm() {
  const router = useRouter()
  const form = useForm<PersonaInput>({
    resolver: zodResolver(personaSchema),
    defaultValues: { tone: 'formal' }
  })

  const onSubmit = async (data: PersonaInput) => {
    const res = await createPersonaAction(data)
    if (!res.ok) { toast.error(res.error); return }
    router.replace('/chat')
    router.refresh()
  }

  return (/* shadcn Form */)
}

```

Server Action

```

// app/(app)/onboarding/actions.ts
'use server'
import { db } from '@lib/db'

```

```

import { personas } from '@/lib/db/schema'
import { getCurrentUser } from '@/lib/auth'
import { personaSchema } from './schema'

export async function createPersonaAction(input: unknown) {
  const parsed = personaSchema.safeParse(input)
  if (!parsed.success) return { ok: false as const, error:
    '입력값을 확인해주세요' }

  const user = await getCurrentUser()
  if (!user) return { ok: false as const, error: '로그인이
    필요합니다' }

  await db.insert(personas).values({
    userId: user.id,
    nickname: parsed.data.nickname,
    ageRange: parsed.data.ageRange,
    tone: parsed.data.tone,
    selfIntro: parsed.data.selfIntro ?? null,
  }).onConflictDoUpdate({
    target: personas.userId,
    set: {
      nickname: parsed.data.nickname,
      ageRange: parsed.data.ageRange,
      tone: parsed.data.tone,
      selfIntro: parsed.data.selfIntro ?? null,
      updatedAt: new Date(),
    }
  })

  return { ok: true as const }
}

```

7.4 채팅 `/chat`

진입 RSC

```

// app/(app)/chat/page.tsx
import { redirect } from 'next/navigation'
import { db } from '@/lib/db'
import { conversations, messages } from '@/lib/db/schema'
import { eq, and, isNull, desc, asc } from 'drizzle-orm'
import { ChatClient } from './chat-client'
import { getCurrentUser, getCurrentPersona } from '@/lib/auth'

export default async function ChatPage() {
  const user = await getCurrentUser()
  if (!user) redirect('/login')

  const persona = await getCurrentPersona()
  if (!persona) redirect('/onboarding')

  const active = await db.query.conversations.findFirst({
    where: and(eq(conversations.userId, user.id), isNull(conversations.endedAt)),
    orderBy: [desc(conversations.startedAt)]
  })

  const history = active
    ? await db.query.messages.findMany({
        where: eq(messages.conversationId, active.id),
        orderBy: [asc(messages.createdAt)]
      })
    : []

  // ★ initialMessages는 useChat의 표준 필드만, 위기 플래그는 별도로 추출
  const initialMessages = history.map(m => ({
    id: String(m.id),
    role: m.role as 'user' | 'assistant',
    content: m.content,
  }))
  const initialCrisisIds = history
    .filter(m => m.crisisFlag)

```

```

    .map(m => String(m.id))

    return (
      <ChatClient
        persona={persona}
        initialMessages={initialMessages}
        initialCrisisIds={initialCrisisIds}
        conversationId={active?.id ?? null}
      />
    )
  }
}

```

`data: { crisis: ... }` 같은 임의 필드를 `initialMessages`에 박지 않는다. **AI SDK** `Message` 표준 필드 밖의 키는 보존 보장 없음. crisis 플래그는 별도 Set으로 관리.

7.5 약관 `/terms` · 개인정보 `/privacy`

정적 RSC. 수집/보관/제3자 제공/의료기기 아님 고지/1393/삭제 요청 채널.

8. 채팅 UI 상세

8.1 `useChat` + 보조 상태

```

// app/(app)/chat/chat-client.tsx
'use client'
import { useChat } from '@ai-sdk/react'
import { useRouter } from 'next/navigation'
import { useState, useTransition, useRef, useEffect } from 'react'
import { toast } from 'sonner'
import { MessageList } from '@components/chat/message-list'
import { ChatInput } from '@components/chat/chat-input'
import { EmptyState } from '@components/chat/empty-state'
import { EndConversationButton } from '@components/chat/end-conversation-button'
import { endConversationAction } from './actions'

```



```

type Msg = { id: string; role: 'user' | 'assistant'; content: string }
type Props = {
  persona: { nickname: string; tone: 'formal' | 'casual' }
  initialMessages: Msg[]
  initialCrisisIds: string[]
  conversationId: string | null
}

export function ChatClient({ persona, initialMessages, initialCrisisIds, conversationId }: Props) {
  const router = useRouter()
  // ★ conversationId state (BE가 새로 생성하면 응답 헤더로 받아 갱신)
  const [activeId, setActiveId] = useState<string | null>(conversationId)
  // ★ 위기 카드 표시 ID 집합 (영속 + 활성 stream 통합)
  const [crisisIds, setCrisisIds] = useState<Set<string>>(() => new Set(initialCrisisIds))
  // 현재 활성 stream이 위기? (ally alert용 - 1회성)
  const [liveCrisis, setLiveCrisis] = useState(false)
  const [isEnding, startEnding] = useTransition()

  const { messages, input, handleInputChange, handleSubmit, status } = useChat({
    api: '/api/chat',
    body: { conversationId: activeId }, // ★ 매 호출 최신 activeId
    initialMessages,
    onResponse(res) {
      // 위기 신호
      if (res.headers.get('x-crisis') === '1') setLiveCrisis(true)
      // ★ BE가 새 conversation을 생성했다면 헤더로 ID 수신
      const newId = res.headers.get('x-conversation-id')
      if (newId && newId !== activeId) setActiveId(newId)
    },
  })

```

```

    onFinish(message) {
      // 마지막 user 메시지가 위기였다면 그 ID를 crisisIds에 추가
      if (liveCrisis) {
        // useChat의 messages 배열에서 직전 user 메시지 찾아 추
가
        const lastUserId = lastUserIdRef.current
        if (lastUserId) {
          setCrisisIds(prev => {
            const next = new Set(prev); next.add(lastUserI
d); return next
          })
        }
      },
      onError() {
        toast.error('응답을 가져오지 못했어요. 잠시 후 다시 시도해주
세요.')
      },
    })

    // 마지막 user 메시지 ID 추적 (위기 ID 영속화용)
    const lastUserIdRef = useRef<string | null>(null)
    useEffect(() => {
      const lu = [...messages].reverse().find(m => m.role ===
'user')
      if (lu) lastUserIdRef.current = lu.id
    }, [messages])

    // ★ wrap된 submit: 새 user 메시지 보낼 때 liveCrisis rese
t
    const onSubmit = (e: React.FormEvent) => {
      setLiveCrisis(false) // 직전 stream의 신호 초기화
      handleSubmit(e)
    }

    const handleEnd = () => {
      if (!activeId) return
      startEnding(async () => {

```

```

    const res = await endConversationAction(activeId)
    if (!res.ok) { toast.error('정리에 실패했어요. 다시 시도
해주세요.');
```

```

    }; return }
    setActiveId(null)
    setCrisisIds(new Set())
    setLiveCrisis(false)
    router.refresh() // RSC가 새 active conversation (없
음) 으로 page 재실행
  })
}

const isEmpty = messages.length === 0

return (
  <div className="flex flex-col h-dvh">
    <header className="border-b px-4 py-3 flex items-cent
er justify-between">
      <span className="font-bold">gohae</span>
      <span className="text-text-secondary text-sm">{pers
ona.nickname}</span>
    </header>

    <main className="flex-1 overflow-y-auto">
      {isEmpty
        ? <EmptyState nickname={persona.nickname} tone={p
ersona.tone} />
        : <MessageList
            messages={messages}
            crisisIds={crisisIds}
            liveCrisisForLastUser={liveCrisis}
          />
      }
    </main>

    <footer
      className="border-t p-3 flex gap-2 items-end"
      style={{ paddingBottom: 'max(0.75rem, env(safe-area
-inset-bottom))' }}
    >

```

```

    >
    <ChatInput
      value={input}
      onChange={handleInputChange}
      onSubmit={onSubmit}
      disabled={status === 'streaming' || isEnding}
    />
    <EndConversationButton
      onClick={handleEnd}
      disabled={!activeId || isEnding || status === 'streaming'}
      isPending={isEnding}
    />
  </footer>
</div>
)
}

```

핵심 결정 정리

항목	처리
새 conversation ID 동기화	<code>onResponse(res)</code> 에서 <code>x-conversation-id</code> 헤더 수신 → <code>setActiveId(newId)</code> . BE TechSpec §6.1에 응답 헤더 명세 필요 (cross-link)
위기 카드 표시 데이터	<code>useChat.data</code> 의존 금지 → 별도 <code>crisisIds: Set<string></code> state. 초기값은 RSC에서 받은 <code>initialCrisisIds</code>
위기 ID 영속화 흐름	<code>onResponse</code> 에서 <code>x-crisis</code> 수신 → <code>liveCrisis=true</code> → <code>onFinish</code> 에서 직전 user 메시지 ID를 <code>crisisIds</code> 에 추가
위기 활성 신호 reset	새 user 메시지 submit 직전 <code>setLiveCrisis(false)</code> (<code>onSubmit</code> wrapper)
대화 종료	Server Action 호출 → 성공 시 <code>activeId=null</code> , <code>crisisIds</code> 비우기 , <code>liveCrisis=false</code> , <code>router.refresh()</code>

8.2 메시지 표시 규칙

role	위치	스타일	위기 카드
user	오른쪽	bg-accent text-accent-fg	crisisIds.has(message.id) 면 직 후 <CrisisCard liveAlert= {crisisIds.has(id) && liveCrisisForLastUser && 마지막 user 메시지} />
assistant	왼쪽	bg-bg-surface border	—

MessageList 시그니처:

```
type Props = {
  messages: { id: string; role: 'user'|'assistant'; conten
t: string }[]
  crisisIds: Set<string>
  liveCrisisForLastUser: boolean
}
```

sentinel `__CRISIS__` 또는 `data.crisis` 같은 message 내장 필드에 의존하지 않는다.
위기 카드는 메시지 ID로 매핑한 외부 Set으로만 결정.

8.3 빈 상태 (Empty State)

```
// components/chat/empty-state.tsx
export function EmptyState({ nickname, tone }: { nickname:
string; tone: 'formal'|'casual' }) {
  const greeting = tone === 'formal'
    ? `${nickname}님, 어서오세요.\n오늘은 어떤 이야기를 들려주실래
요?`
    : `${nickname}아, 안녕.\n오늘은 무슨 이야기 할까?`
  return (
    <div className="h-full flex flex-col items-center justi
fy-center text-center px-8 text-text-secondary">
      <p className="whitespace-pre-line leading-relaxed">{g
reeting}</p>
    </div>
  )
}
```

- 첫 진입·대화 종료 직후 모두 사용.
- 사용자가 첫 메시지 보내면 BE가 자동으로 새 conversation 생성 → 응답 헤더로 ID 회신 → `activeId` 갱신.

8.4 "대화 끝내기" UX

단계	상태	UI
클릭 가능	<code>activeId</code> 있고 streaming 아님	버튼 활성화
클릭 직후	<code>useTransition</code> pending	버튼 "정리하는 중..." + 스피너, 입력창 비활성
성공	<code>activeId/crisisId</code> s reset + <code>router.refresh()</code>	메시지 클리어, EmptyState
실패	sonner toast	버튼 활성화 복귀
새 메시지 입력	<code>activeId === null</code> 상태로 <code>/api/chat</code> 호출	BE가 새 conversation 생성, 응답 헤더로 ID 회신

Server Action — 직접 호출 (fetch 금지)

```
// app/(app)/chat/actions.ts
'use server'
import { revalidatePath } from 'next/cache'
import { and, eq, isNull, asc } from 'drizzle-orm'
import * as Sentry from '@sentry/nextjs'
import { db } from '@/lib/db'
import { conversations, messages } from '@/lib/db/schema'
import { getCurrentUser } from '@/lib/auth'
import { extractAndStoreMemories } from '@/lib/ai/memory/extract'

export const runtime = 'nodejs'

export async function endConversationAction(conversationId: string) {
  const user = await getCurrentUser()
  if (!user) return { ok: false as const }

  // 원자적 종료 (BE TechSpec §6.2와 동일 패턴)
```

```

const claimed = await db.update(conversations)
  .set({ endedAt: new Date() })
  .where(and(
    eq(conversations.id, conversationId),
    eq(conversations.userId, user.id),
    isNull(conversations.endedAt),
  ))
  .returning({ id: conversations.id })

if (claimed.length === 0) {
  revalidatePath('/chat')
  return { ok: true as const, alreadyEnded: true }
}

// 메시지 수집 + 추출
const msgs = await db.query.messages.findMany({
  where: eq(messages.conversationId, conversationId),
  orderBy: [asc(messages.createdAt)]
})

try {
  await extractAndStoreMemories(user.id, msgs)
  await db.update(conversations)
    .set({ summaryStatus: 'completed' })
    .where(eq(conversations.id, conversationId))
} catch (e) {
  Sentry.captureException(e)
  await db.update(conversations)
    .set({ summaryStatus: 'failed' })
    .where(eq(conversations.id, conversationId))
  // 추출 실패해도 대화 종료는 유지
}

revalidatePath('/chat')
return { ok: true as const }
}

```



❌ 절대 이렇게 하지 말 것 (이전 버전 함정)

`fetch(`${SITE_URL}/api/conversation/end`, { headers: { cookie: '' } })` —

Server Action에서 자기 자신의 API route를 fetch하면 빈 cookie로 인증 실패. v1.0은 Server Action 안에서 `extractAndStoreMemories` 를 직접 호출.

`/api/conversation/end` route는 외부/모바일 호출용으로 BE TechSpec에 남아 있을 수 있지만 v1.0 FE는 사용하지 않음.

8.5 스트리밍 UX

- 응답 중 입력창 disabled.
- 자동 스크롤: `useEffect` 로 `messages.length` 변화 시 `scrollIntoView({ behavior: 'smooth' })`.
- 응답 중단 버튼: v1.0 out.
- 마크다운: BE 시스템 프롬프트가 "plain text" 지시 — FE는 `whitespace-pre-line` 으로 줄바꿈 보존.

9. 위기 안내 카드 — 사이클 명세 포함



원칙 — 위기 신호는 두 채널. (a) **영속**: `messages.crisis_flag` 컬럼 → RSC에서 추출하여 `initialCrisisIds` prop으로 전달 → 클라이언트 `crisisIds: Set` 초기화. (b) **활성 stream**: 응답 헤더 `x-crisis: 1` → `liveCrisis=true` → `onFinish` 에서 직전 user 메시지 ID를 `crisisIds` 에 추가. **메시지 content에 sentinel 없음**. `useChat.data` 같은 비표준 메시지 필드에 의존 안 함.

9.1 흐름 및 reset 사이클

flowchart TB

A["RSC: history 로드"] --> B["crisisIds = Set(history에서 crisis_flag=true인 id)"]

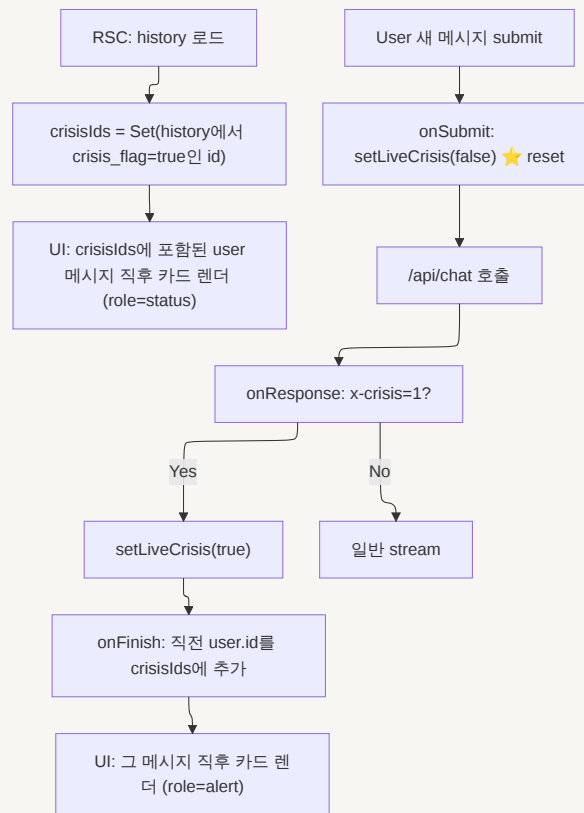
B --> C["UI: crisisIds에 포함된 user 메시지 직후 카드 렌더 (role=status)"]

D["User 새 메시지 submit"] --> E["onSubmit: setLiveCrisis(false) ★ reset"]


```

E --> F["/api/chat 호출"]
F --> G["onResponse: x-crisis=1?"]
G -->|"Yes"| H["setLiveCrisis(true)"]
G -->|"No"| I["일반 stream"]
H --> J["onFinish: 직전 user.id를 crisisIds에 추가"]
J --> K["UI: 그 메시지 직후 카드 렌더 (role=alert)"]

```



9.2 <CrisisCard />

```

// components/chat/crisis-card.tsx
type Props = { liveAlert: boolean } // true: 활성 stream 도
착 / false: 복원분

export function CrisisCard({ liveAlert }: Props) {
  return (
    <div
      role={liveAlert ? 'alert' : 'status'}
      aria-live={liveAlert ? 'assertive' : 'polite'}
      className="my-3 rounded-md border border-danger/30 bg

```

```

-danger-bg p-4"
  >
    <p className="font-medium text-danger mb-2">잠시만요</
p>
    <p className="text-sm leading-relaxed text-text-primary">
      지금 많이 힘드신 것 같아요. 혼자 감당하기 어려운 마음이 들
      땐, 전문 상담사와 익명으로 이야기할 수 있는 곳이 있어요.
    </p>
    <a
      href="tel:1393"
      className="mt-3 inline-flex items-center gap-1 font-
-medium text-danger underline"
    >
      자살예방상담전화 1393 (24시간, 무료)
    </a>
  </div>
)
}

```

custom div (shadcn Alert 미사용).

9.3 키워드 매칭 (`lib/crisis/keywords.ts`)

```

export const CRISIS_KEYWORDS = [
  '자살', '자해', '죽고 싶', '사라지고 싶', '없어지고 싶',
  '목매', '뛰어내리', '약을 먹고', '극단적 선택',
  '끝내고 싶', '사라질래', '그만하고 싶',
  // T+0:00~0:30 구간 외부 자료 참고 확장
] as const

export function containsCrisisSignal(text: string): boolean
{
  const normalized = text.toLowerCase()
  return CRISIS_KEYWORDS.some(k => normalized.includes(k))
}

```



한계 인정 — substring 매칭은 한국어 변형에 false negative. v1.0은 **false positive** 쪽으로 보수적. LLM 2nd-pass 분류는 v1.x.

10. 에러 처리

레벨	처리
API 호출 실패 (<code>useChat onError</code>)	sonner 토스트, Sentry 자동 캡처
Server Action 실패	<code>{ ok: false, error: string }</code> 리턴, 폼/토스트
Route Boundary (<code>app/error.tsx</code>)	"잠시 후 다시 시도" + Sentry capture
Global (<code>app/global-error.tsx</code>)	RSC 실패 fallback
404 (<code>app/not-found.tsx</code>)	홈 링크

```
// app/error.tsx
'use client'
import * as Sentry from '@sentry/nextjs'
import { useEffect } from 'react'
import { Button } from '@components/ui/button'

export default function Error({ error, reset }: { error: Error; reset: () => void }) {
  useEffect(() => { Sentry.captureException(error) }, [error])
  return (
    <div className="flex h-dvh items-center justify-center text-center">
      <div>
        <p className="text-text-secondary mb-4">잠시 문제가 발생했어요</p>
        <Button onClick={reset}>다시 시도</Button>
      </div>
    </div>
  )
}
```

11. Sentry FE — exception scrubbing 강화



민감정보 보호 규칙 — `throw new Error('메시지 전송 실패: ' + userText)` 같은 패턴은 `error.message`에 사용자 메시지가 그대로 들어가 Sentry로 전송됨. 정신건강 도메인에서 절대 금지. **에러 메시지에 사용자 입력을 인터폴레이트하지 않는다.** ID·코드·길이 같은 메타만 포함.

11.1 설치

```
npx @sentry/wizard@latest -i nextjs
```

11.2 scrubbing (request data + exception value)

```
// sentry.client.config.ts
import * as Sentry from '@sentry/nextjs'

// 한국어 50자 이상의 연속된 텍스트가 보이면 redact (휴리스틱)
const looksLikeUserMessage = (s: string) =>
  /[가-힣\s.,!~]{30,}/.test(s)

Sentry.init({
  dsn: process.env.NEXT_PUBLIC_SENTRY_DSN,
  tracesSampleRate: 0.1,
  beforeSend(event) {
    // 1) request.data 스크러빙
    if (event.request?.data && typeof event.request.data ==
    = 'object') {
      const data = event.request.data as Record<string, unknown>
      if ('messages' in data) data.messages = '[REDACTED]'
      if ('content' in data) data.content = '[REDACTED]'
    }
    // 2) exception value 스크러빙 - error.message에 인터폴레이트된
    사용자 입력 차단
    if (event.exception?.values) {
      for (const ex of event.exception.values) {
```

```

        if (ex.value && looksLikeUserMessage(ex.value)) {
            ex.value = '[REDACTED - possible user content]'
        }
    }
}
return event
},
beforeBreadcrumb(crumb) {
    if (crumb.category === 'fetch' && (crumb.data?.url as string)?.includes('/api/chat')) {
        if (crumb.data) delete (crumb.data as any).body
    }
    return crumb
},
})

```

server config(`sentry.server.config.ts`)에도 동일 `beforeSend` 적용 — BE TechSpec §12 참조.

12. SEO / 메타데이터

```

// app/layout.tsx
export const metadata: Metadata = {
    title: { default: 'gohae - 나를 기억하는 대화', template: '%s · gohae' },
    description: '매번 처음부터 설명하지 않아도 괜찮아요. 고해는 당신의 맥락을 이어가는 AI 상담 서비스입니다.',
    openGraph: { title: 'gohae', description: '나를 기억하는 대화', locale: 'ko_KR', type: 'website' },
    robots: { index: true, follow: true },
}

```

채팅/온보딩은 `robots: { index: false }`.

13. 접근성

- 키보드 포커스 (shadcn 기본).
- 폼 label 연결.
- 위기 카드: **활성 stream 도착분** `role="alert"`, **복원분** `role="status"` (§9.2 `liveAlert`).
- 색상 대비 AAA (16.8:1).
- 폰트 최소 14px.

14. 모바일 / 반응형

항목	정책
채팅 화면 높이	<code>h-dvh</code>
키보드	<code>interactiveWidget: 'resizes-content'</code> viewport meta
하단 input padding	<code>paddingBottom: max(0.75rem, env(safe-area-inset-bottom))</code>
입력 폰트	16px 이상 (iOS zoom 방지)

```
// app/layout.tsx
export const viewport: Viewport = {
  width: 'device-width',
  initialScale: 1,
  interactiveWidget: 'resizes-content',
  themeColor: '#FAFAF7',
}
```

15. 성능

- 폰트: `next/font/google` Noto_Sans_KR, weight [400,500,700], display 'swap'.
- 이미지: 랜딩 한정 `next/image`.
- RSC 활용으로 클라이언트 번들 최소화.
- 채팅 가상화 v1.0 out.

16. 테스트 (v1.0 수동)

골든패스	검증
가입 → 온보딩 → 첫 채팅 → 끝내기 → 새 메시지 → 이전 기억 반영	메모리 시스템
첫 메시지 직후 응답 헤더에 <code>x-conversation-id</code> 도착 + <code>activeId</code> 갱신	conversationId 동기화
위기 키워드 메시지 → 1393 카드 (live, role=alert)	활성 stream 위기
새로고침 후 위기 카드 복원 (role=status)	crisisIds 영속화
위기 메시지 → 다음 일반 메시지 보낼 때 liveCrisis reset 확인	사이클 정합
재방문 (persona 있는 계정 로그인) → <code>/chat</code> 직행	callback 분기
/onboarding URL 직접 진입 (persona 있는 상태) → <code>/chat</code> redirect	onboarding 가드
모바일 Safari 키보드 올라올 때 입력창 가시성	viewport meta
<code>data</code> 필드 의존 제거 검증 — initialMessages에 data 키 안 넘기는 코드 확인	AI SDK 호환성

17. 환경변수 (FE에서 사용)

```

NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
NEXT_PUBLIC_SENTRY_DSN=
NEXT_PUBLIC_SITE_URL=https://gohae.vercel.app

```

서버 변수는 BE TechSpec §11.



BE TechSpec과 cross-link 필수 항목

- `messages.status` enum (BE §3.1) — 새로고침 시 placeholder(`pending`)/실패(`failed`) 메시지 표시 UX는 FE v1.x에서 보강 (v1.0은 단순 plain content 렌더).
- `/api/chat` 응답 헤더 `x-conversation-id` — BE TechSpec §6.1에 추가 명세 필요. FE는 `onResponse` 에서 수신 가정.
- 모든 Drizzle 사용 route handler `runtime = 'nodejs'` — `/api/chat`, `/auth/callback`, 모든 Server Action.
- Sentry exception scrubbing은 server config에도 동일 적용 (BE §12).

18. 미해결 결정 (v1.x로 이관)

- ☐ 다크모드.
- ☐ 채팅 입력창 자동 높이.
- ☐ 마크다운 렌더.
- ☐ PWA / 모바일 앱.
- ☐ 위기 시그널을 Vercel AI SDK **message annotations**로 이관 (현재 응답 헤더 + 클라이언트 Set).
- ☐ 위기 감지 LLM 2nd-pass 분류.
- ☐ 응답 중단 버튼.
- ☐ `messages.status` 별 UI 분기 (placeholder pending, failed 표시).

19. 결정 로그

일자	결정	근거
@2026년 5월 16일	Tailwind v4 + shadcn (new-york)	12h 스프린트 빠른 진행
@2026년 5월 16일	RSC + Server Actions 우선, 글로벌 상태 라이بر러리 없음	Next.js 15 권장
@2026년 5월 16일	채팅: Vercel AI SDK <code>useChat</code>	스트리밍·상태 일체
@2026년 5월 16일	위기 신호 채널: <code>messages.crisis_flag</code> (영속) + <code>x-crisis</code> 헤더 (활성)	sentinel 안티패턴 회피
@2026년 5월 16일	위기 카드 표시 데이터: <code>crisisIds: Set<string></code> 별도 state	useChat의 비표준 <code>data</code> 필드 의존 회피
@2026년 5월 16일	<code>crisisActive</code> reset: 새 user 메시지 submit 시점	활성 신호의 명확한 사이클
@2026년 5월 16일	OAuth callback persona 분기 + onboarding/page 이중 가드	depth-in-defense
@2026년 5월 16일	middleware는 세션 refresh + 비로그인 차단 만. DB 조회 없음	Edge runtime 부담
@2026년 5월 16일	<code>(app)/layout.tsx</code> 는 인증만 (persona 조회 제거)	cache 우회·의미 없는 조회 차단
@2026년 5월 16일	hard navigation 대신 <code>router.replace + router.refresh</code>	Next.js 정합

일자	결정	근거
@2026년 5월 16일	<code>NEXT_PUBLIC_SITE_URL</code> OAuth redirect URI 명시적 사용	Vercel preview에서도 production
@2026년 5월 16일	위기 카드 a11y 분기 (<code>liveAlert</code>)	스크린리더 중복 방지
@2026년 5월 16일	대화 끝내기: 로딩 → revalidate → EmptyState	사용자 인식 가능한 변화
@2026년 5월 16일	모바일: <code>h-dvh</code> • <code>interactiveWidget</code> • <code>safe-area-inset</code>	iOS Safari 100vh / 키보드
@2026년 5월 16일	키워드 매칭 false negative 한계 인정, LLM 2nd-pass v1.x	12h 우선순위
@2026년 5월 16일	자동화 테스트 v1.0 out	스프린트 시간 보호
@2026년 5월 16일	Sentry scrubbing: request body + exception value (휴리스틱)	메시지 인터폴레이트된 에러 방어
@2026년 5월 16일	에러 메시지에 사용자 입력 인터폴레이트 금지 규칙	정신건강 도메인 민감도
@2026년 5월 16일	폼: react-hook-form + zod	shadcn 권장
@2026년 5월 16일	메모리 추출은 Server Action에서 <code>extractAndStoreMemories</code> 직접 호출 (fetch 우회 함정 회피)	cookie 전달 문제 + 동일 프로세스
@2026년 5월 16일	새 conversationId는 응답 헤더 <code>x-conversation-id</code> • <code>useState(activeId)</code> 동기화	BE 생성한 ID를 클라이언트로 전달하는 단일 경로
@2026년 5월 16일	<code>getCurrentPersona</code> 는 정적 import (dynamic import 제거)	가독성·유지보수, tree-shaking 동일
@2026년 5월 16일	shadcn <code>alert</code> 컴포넌트 미설치 (CrisisCard 는 custom div)	단순성
@2026년 5월 16일	Drizzle 쓰는 모든 route handler <code>runtime = 'nodejs'</code>	postgres-js Edge 미동작



다음 액션 — BE TechSpec 에 응답 헤더 `x-conversation-id` 명세를 추가하면 정합 완료. 부트스트랩 슬롯(T+1:30~2:30) 진입.